

|- entidade.entity.ts

| - entidade.controller.ts

| - entidade.model.ts

| - entidade.service.ts

Para cada uma das entidades.

Para executar o projeto:

npm install

npm run start

2. Arquitetura do Software

Arquitetura e principais tecnologias

Frontend:

- Typescript e React

Backend:

- Node.js com Typescript

Banco de Dados:

- PostgreSQL como banco de dados relacional.

Testes:

Backend:

- Nest.js (tem seu próprio esquema de teste)
- Swagger (teste de end point e documentação)

Frontend:

- Swagger
- Jest

A arquitetura proposta para esse projeto é a arquitetura em camadas.

Camada de módulo: Responsável por organizar e agrupar as funcionalidades da entidade.

Camada de controllers: Responsável por lidar com as requisições HTTP da API.

Camada de services: Responsável por conter a lógica de negócios da aplicação.

Camada de entity: Responsável por criar a estrutura de dado padrão das entidades.

Camada de DTO: Responsável por ter a estrutura dos objetos no formato que serão passados e transferidos dentro da aplicação.

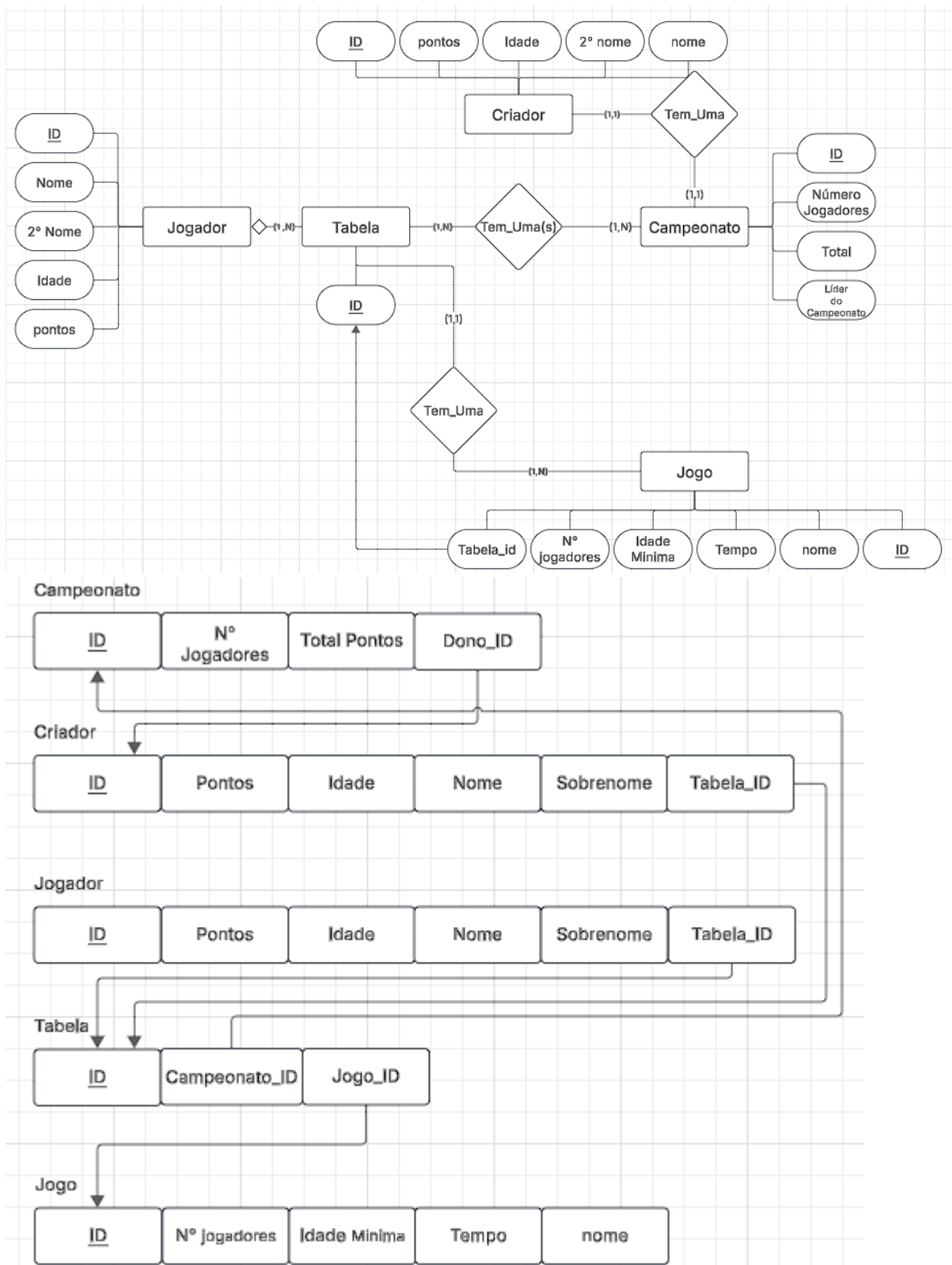
Camada de teste: Responsável por testar casos de uso, tentar captar possíveis falhas no código decorrentes do desenvolvimento.

Camada do Banco de dados: Responsável por armazenar todas as informações que têm que ser persistidas e armazenadas por muito tempo.

Camada de front end: Essa camada é a interface com o usuário. Ela é responsável por apresentar os dados e permitir que o usuário interaja com a aplicação.

3. Definição de Entidades e Relacionamentos

Imagem que detalha os relacionamentos dentre todas as entidades e suas características.



4. Conexão com o Banco de Dados

O acesso ao bando de dados é feito utilizando de tecnologias como sequelize (para organizar a interação com banco de dados) e express (para organizar as requisições HTTP) e seus padrões apresentados em sala.

Banco de Dados Utilizado: PostgreSQL (banco de dados relacional muito visto em projetos de maior porte)

Itens presentes:

Tabela de criador.

Tabela de jogador.

Tabela de campeonato.

Tabela de jogo.

Tabela de tabela (responsável por contabilizar os pontos da rodada e do campeonato).

Para essa entrega apenas as funções de crud de cada entidade.

5. Projeto Rodando e Fazendo Requisição HTTP

Endpoints Implementados:

Campeonato:

Post/api/campeonato: cria o campeonato

Get/api/campeonato: lista os jogadores do campeonato

Get/api/campeonato/:id: acha um jogador do campeonato

Patch/api/campeonato/:id: atualiza um certo campeonato

Delete/api/campeonato/:id: deleta o campeonato

Criador:

Post/api/criador: cria o criador

Get/api/criador: lista todos os jogadores que estão em campeonatos do criador

Get/api/criador:id: acha um jogador que está em algum campeonato criado pelo criador

Patch/api/criador:id: atualiza dados do criador

Delete/api/criador:id: deleta o criador

Jogador:

Post/api/jogador: cria o jogador

Get/api/jogador: lista todos os jogadores no banco de dados

Get/api/jogador:id: acha um jogador em todo o banco de dados

Patch/api/jogador:id: atualiza dados de um jogador

Delete/api/jogador: deleta o jogador

Jogo:

Post/api/jogo: cria o jogo

Get/api/jogo: lista todos os jogos

Get/api/jogo:id: acha um jogo no banco de dados

Patch/api/jogo:id atualiza dados do jogo

Delete/api/jogo: deleta o jogo

Tabela:

Post/api/tabela: cria a tabela

Get/api/tabela: lista todas as tabelas de todos os campeonatos já realizados

Get/api/tabela:id: acha a tabela do campeonato requerido

Patch/api/tabela:id: atualiza a tabela

Delete/api/tabela: deleta a tabela.